



Guía Completa: Despliegue VPS Contabo con Multi-Apps

De Docker a Producción: Django Ecodisseny + Oscar Shop + VSCode Remote

Índice Completo



FASE 1: PREPARACIÓN

- [1.1 Preparación Pre-Migración](#)
- [1.2 Backup y Configuración](#)



FASE 2: VPS CONTABO

- [2.1 Contratación y Especificaciones](#)
- [2.2 Configuración Inicial del Sistema](#)
- [2.3 Instalación de Dependencias](#)



FASE 3: MIGRACIÓN DE DATOS

- [3.1 Clonación del Repositorio](#)
- [3.2 Configuración con Scripts Automatizados](#)
- [3.3 Transferencia de Datos](#)



FASE 4: CONFIGURACIÓN DE PRODUCCIÓN

- [4.1 Django Settings de Producción](#)
- [4.2 Nginx y Proxy Reverso](#)
- [4.3 Gunicorn como Servicio](#)



FASE 5: DNS Y SSL

- [5.1 Configuración DNS Cloudflare](#)
- [5.2 Certificados SSL Let's Encrypt](#)
- [5.3 Optimizaciones Cloudflare](#)



FASE 6: DESARROLLO REMOTO

- [6.1 VSCode Remote SSH](#)
- [6.2 Configuración de Desarrollo](#)
- [6.3 Debugging y Testing](#)



FASE 7: MULTI-APLICACIONES

- [7.1 Arquitectura Multi-App](#)
- [7.2 Configuración Django Oscar](#)
- [7.3 Gestión de Puertos](#)

 FASE 8: MONITOREO Y MANTENIMIENTO

- [8.1 Scripts de Monitoreo](#)
- [8.2 Backups Automatizados](#)
- [8.3 Scripts de Actualización](#)

 FASE 9: TROUBLESHOOTING

- [9.1 Problemas Comunes](#)
- [9.2 Debugging Avanzado](#)
- [9.3 Checklist de Verificación](#)

 FASE 1: PREPARACIÓN

1.1 Preparación Pre-Migración

Checklist de Verificación

```
# En tu entorno Docker actual:
✓ Verificar que la aplicación funciona correctamente
✓ Backup completo de la base de datos
✓ Backup de archivos media
✓ Documentar configuraciones específicas
✓ Preparar variables de entorno de producción
✓ Validar que todos los tests pasan
```

1.2 Backup y Configuración

Generar Backup Completo

```
# Asegúrate de que Docker está funcionando
docker-compose up -d

# Backup de base de datos
docker-compose exec db pg_dump -U postgres ecodisseny >
backup_desarrollo_$(date +%Y%m%d).sql

# Backup de archivos media
tar -czf media_backup_$(date +%Y%m%d).tar.gz media/

# Listar apps y dependencias
docker-compose exec web pip freeze > requirements_frozen.txt
```

Preparar Variables de Entorno

```
# Crear archivo .env de producción
cat > .env.production << EOF
DEBUG=False
SECRET_KEY=GENERAR_UNA_CLAVE_SUPER_SECRETA_AQUI_64_CARACTERES_MINIMO
DATABASE_URL=postgresql://deploy:PASSWORD_SEGURO@localhost:5432/ecodisseny_
prod
ALLOWED_HOSTS=app.arasmu.net,tu-ip-vps
SECURE_SSL_REDIRECT=True
SECURE_PROXY_SSL_HEADER=HTTP_X_FORWARDED_PROTO,https
EOF
```

FASE 2: VPS CONTABO

2.1 Contratación y Especificaciones

VPS Contabo Cloud VPS 10

```
📄 Especificaciones:
├─ CPU: 3 vCPU Cores
├─ RAM: 8 GB
├─ Storage: 75 GB NVMe
├─ Región: European Union
├─ Precio: €3.60/mes (€43.20/año)
└─ Ancho de banda: 32TB

🎯 Capacidad vs Uso Proyectado:
├─ Django Ecodisseny: ~200MB RAM
├─ Django Oscar Shop: ~200MB RAM
├─ PostgreSQL: ~300MB RAM
├─ Nginx + Sistema: ~500MB RAM
└─ Total estimado: ~1.2GB de 8GB (sobra mucho!)
```

2.2 Configuración Inicial del Sistema

Primer Acceso y Seguridad

```
# Conectar por primera vez (Contabo te enviará credenciales)
ssh root@TU_IP_VPS

# Actualizar sistema
apt update && apt upgrade -y

# Crear usuario deploy
useradd -m -s /bin/bash deploy
usermod -aG sudo deploy
passwd deploy
```

```
# Configurar SSH para deploy
mkdir -p /home/deploy/.ssh
echo "tu_clave_publica_ssh" >> /home/deploy/.ssh/authorized_keys
chown -R deploy:deploy /home/deploy/.ssh
chmod 700 /home/deploy/.ssh
chmod 600 /home/deploy/.ssh/authorized_keys

# Configurar firewall
ufw allow 22      # SSH
ufw allow 80      # HTTP
ufw allow 443     # HTTPS
ufw --force enable

# Cambiar a usuario deploy
su - deploy
```

2.3 Instalación de Dependencias

```
# Instalar dependencias del sistema
sudo apt install -y \
    python3 \
    python3-pip \
    python3-venv \
    python3-dev \
    build-essential \
    libpq-dev \
    postgresql \
    postgresql-contrib \
    nginx \
    certbot \
    python3-certbot-nginx \
    htop \
    curl \
    wget \
    unzip

# Configurar PostgreSQL
sudo -u postgres createuser --interactive deploy
sudo -u postgres createdb -O deploy ecodisseny_prod
sudo -u postgres psql -c "ALTER USER deploy PASSWORD
'tu_password_postgresql_seguro';"

# Crear estructura de directorios
mkdir -p /home/deploy/{ecodisseny,oscar_shop,scripts,backups,logs}
mkdir -p /var/log/django
sudo chown deploy:deploy /var/log/django
```

3.1 Clonación del Repositorio

```
cd /home/deploy/ecodisseny

# Clonar desde GitHub
git clone https://github.com/Mulastone/ecodisseny_dj_pg.git .

# Crear entorno virtual
python3 -m venv venv
source venv/bin/activate

# Instalar dependencias
pip install -r requirements.txt
pip install gunicorn psycopg2-binary whitenoise
```

3.2 Configuración con Scripts Automatizados

✂ Uso de Scripts de Configuración del Proyecto

```
cd /home/deploy/ecodisseny
source venv/bin/activate

# Configurar variables de entorno
cat > .env << 'EOF'
DEBUG=False
SECRET_KEY=tu_secret_key_super_segura_de_64_caracteres_minimo_aqui
DATABASE_URL=postgresql://deploy:tu_password_postgresql_seguro@localhost:54
32/ecodisseny_prod
ALLOWED_HOSTS=app.arasmu.net,tu-ip-vps
SECURE_SSL_REDIRECT=True
SECURE_PROXY_SSL_HEADER=HTTP_X_FORWARDED_PROTO,https
EOF

# Opción 1: Configuración completa automática (RECOMENDADO)
chmod +x setup_complete.sh
./setup_complete.sh

# Esto ejecuta automáticamente:
# ✔ Reinicio de migraciones
# ✔ Configuración de base de datos
# ✔ Carga de fixtures maestros
# ✔ Creación de usuarios y perfiles

# Usuarios creados automáticamente:
echo "👑 ADMIN - mulastone: ecodisseny2024"
echo "👑 ADMIN - gonzalo: ecodisseny2024"
echo "👤 USER - sarah: ecodisseny2024"
echo "👤 USER - pilar: ecodisseny2024"
echo "👤 USER - santiago: ecodisseny2024"
echo "👤 USER - roger: ecodisseny2024"
```

```
# Cargar documentación y configurar cache
python manage.py cargar_documentacion
python manage.py createcachetable
python manage.py collectstatic --noinput
```

3.3 Transferencia de Datos

```
# En tu máquina local (transferir backups):
scp backup_desarrollo_*.sql deploy@tu-ip-vps:/home/deploy/backups/
scp media_backup_*.tar.gz deploy@tu-ip-vps:/home/deploy/backups/

# En el VPS (restaurar si necesario):
# psql ecodisseny_prod < /home/deploy/backups/backup_desarrollo_*.sql
# tar -xzf /home/deploy/backups/media_backup_*.tar.gz
```



FASE 4: CONFIGURACIÓN DE PRODUCCIÓN

4.1 Django Settings de Producción

```
# Crear ecodisseny/settings_production.py
import os
from .settings import *

# Configuración de producción
DEBUG = False
SECRET_KEY = os.getenv('SECRET_KEY')
ALLOWED_HOSTS = os.getenv('ALLOWED_HOSTS', '').split(',')

# Base de datos
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'ecodisseny_prod',
        'USER': 'deploy',
        'PASSWORD': os.getenv('DB_PASSWORD'),
        'HOST': 'localhost',
        'PORT': '5432',
        'OPTIONS': {
            'MAX_CONNS': 20,
        }
    }
}

# Cache en base de datos (sin Redis para simplicidad)
CACHES = {
    'default': {
        'BACKEND': 'django.core.cache.backends.db.DatabaseCache',
```

```

        'LOCATION': 'django_cache_table',
    }
}

# Archivos estáticos
STATIC_ROOT = '/home/deploy/ecodisseny/static/'
MEDIA_ROOT = '/home/deploy/ecodisseny/media/'

# Configuración de seguridad
SECURE_SSL_REDIRECT = True
SECURE_PROXY_SSL_HEADER = ('HTTP_X_FORWARDED_PROTO', 'https')
SECURE_BROWSER_XSS_FILTER = True
SECURE_CONTENT_TYPE_NOSNIFF = True
X_FRAME_OPTIONS = 'DENY'

# Logging optimizado
LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'formatters': {
        'verbose': {
            'format': '{levelname} {asctime} {module} {process:d}
{thread:d} {message}',
            'style': '{',
        },
    },
    'handlers': {
        'file': {
            'level': 'INFO',
            'class': 'logging.handlers.RotatingFileHandler',
            'filename': '/var/log/django/ecodisseny.log',
            'maxBytes': 10*1024*1024,
            'backupCount': 5,
            'formatter': 'verbose',
        },
    },
    'loggers': {
        'django': {
            'handlers': ['file'],
            'level': 'INFO',
            'propagate': True,
        },
    },
}

```

4.2 Nginx y Proxy Reverso

```

# Configuración Nginx para Ecodisseny
sudo tee /etc/nginx/sites-available/app.arasmu.net << 'EOF'
server {
    listen 80;

```

```
server_name app.arasmu.net;
return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl http2;
    server_name app.arasmu.net;

    # SSL será configurado por Certbot

    # Security headers
    add_header X-Frame-Options DENY always;
    add_header X-Content-Type-Options nosniff always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header Strict-Transport-Security "max-age=31536000;
includeSubDomains" always;

    # Archivos estáticos
    location /static/ {
        alias /home/deploy/ecodisseny/static/;
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    location /media/ {
        alias /home/deploy/ecodisseny/media/;
        expires 30d;
    }

    # Django app
    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_redirect off;
    }

    # Logs
    access_log /var/log/nginx/app.arasmu.net.access.log;
    error_log /var/log/nginx/app.arasmu.net.error.log;
    client_max_body_size 50M;
}
EOF

# Habilitar sitio
sudo ln -s /etc/nginx/sites-available/app.arasmu.net /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl restart nginx
```


4.3 Gunicorn como Servicio

```
# Configuración Gunicorn
mkdir -p /home/deploy/ecodisseny/config
cat > /home/deploy/ecodisseny/config/gunicorn.conf.py << 'EOF'
bind = "127.0.0.1:8000"
workers = 2 # Optimizado para 8GB RAM y pocos usuarios
worker_class = "sync"
timeout = 300
max_requests = 1000
max_requests_jitter = 100
preload_app = True
accesslog = "/var/log/django/gunicorn_access.log"
errorlog = "/var/log/django/gunicorn_error.log"
loglevel = "info"
proc_name = "gunicorn_ecodisseny"
user = "deploy"
group = "deploy"
EOF

# Servicio Systemd
sudo tee /etc/systemd/system/ecodisseny.service << 'EOF'
[Unit]
Description=Ecodisseny Django App
Requires=postgresql.service
After=network.target postgresql.service

[Service]
Type=exec
User=deploy
Group=deploy
WorkingDirectory=/home/deploy/ecodisseny
Environment="PATH=/home/deploy/ecodisseny/venv/bin"
Environment="DJANGO_SETTINGS_MODULE=ecodisseny.settings_production"
EnvironmentFile=/home/deploy/ecodisseny/.env
ExecStart=/home/deploy/ecodisseny/venv/bin/gunicorn -c
/home/deploy/ecodisseny/config/gunicorn.conf.py ecodisseny.wsgi:application
ExecReload=/bin/kill -s HUP $MAINPID
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
EOF

# Activar servicio
sudo systemctl daemon-reload
sudo systemctl enable ecodisseny
sudo systemctl start ecodisseny
sudo systemctl status ecodisseny
```

🌐 FASE 5: DNS Y SSL

5.1 Configuración DNS Cloudflare

Registros DNS a Añadir

```
# En Cloudflare Dashboard > DNS > Records:
```

1 Aplicación Django:

Tipo: A

Nombre: app

Contenido: IP_DE_TU_VPS_CONTABO

Proxy status: DNS only (🟡 gris) - IMPORTANTE

2 Futuro Oscar Shop:

Tipo: A

Nombre: tienda

Contenido: IP_DE_TU_VPS_CONTABO

Proxy status: DNS only (🟡 gris) - IMPORTANTE

3 Administración (opcional):

Tipo: A

Nombre: admin

Contenido: IP_DE_TU_VPS_CONTABO

Proxy status: DNS only (🟡 gris) - IMPORTANTE

Verificar Propagación

```
# Verificar desde terminal local:  
nslookup app.arasmu.net  
dig app.arasmu.net  
# Debe devolver la IP de tu VPS
```

5.2 Certificados SSL Let's Encrypt

```
# Una vez propagado el DNS:  
sudo certbot --nginx -d app.arasmu.net -d tienda.arasmu.net  
  
# Configurar renovación automática  
sudo systemctl enable certbot.timer  
sudo systemctl start certbot.timer  
sudo certbot renew --dry-run
```

5.3 Optimizaciones Cloudflare

```
# Una vez SSL funcionando, opcionalmente cambiar a "Proxied":  
# - Cambiar "DNS only" a "Proxied" en Cloudflare  
# - SSL/TLS mode: "Full (strict)"  
# - Auto Minify: CSS, JS, HTML  
# - Brotli: ON
```

FASE 6: DESARROLLO REMOTO

6.1 VSCode Remote SSH

Configurar SSH Local

```
# En tu máquina local, editar ~/.ssh/config:  
Host ecodisseny-vps  
  HostName tu-ip-vps-contabo  
  User deploy  
  Port 22  
  IdentityFile ~/.ssh/id_rsa  
  ServerAliveInterval 60  
  ServerAliveCountMax 3
```

Conectar VSCode

```
# 1. Instalar extensión "Remote - SSH" en VSCode  
# 2. F1 > "Remote-SSH: Connect to Host"  
# 3. Seleccionar "ecodisseny-vps"  
# 4. ¡Listo! VSCode conectado al VPS
```

6.2 Configuración de Desarrollo

Python Interpreter y Extensiones

```
// En VSCode conectado al VPS:  
{  
  "python.defaultInterpreterPath":  
    "/home/deploy/ecodisseny/venv/bin/python",  
  "python.terminal.activateEnvironment": true,  
  "python.linting.enabled": true,  
  "python.formatting.provider": "black"  
}
```

Aliases Útiles

```
# Añadir a ~/.bashrc del VPS:
echo 'source /home/deploy/ecodisseny/venv/bin/activate' >> ~/.bashrc
echo 'alias cdeco="cd /home/deploy/ecodisseny"' >> ~/.bashrc
echo 'alias runserver="python manage.py runserver 0.0.0.0:8001"' >>
~/.bashrc
echo 'alias logs="sudo tail -f /var/log/django/ecodisseny.log"' >>
~/.bashrc
source ~/.bashrc
```

6.3 Debugging y Testing

Configuración Debug

```
// .vscode/launch.json en el VPS:
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Django Debug (Development Mode)",
      "type": "python",
      "request": "launch",
      "program": "${workspaceFolder}/manage.py",
      "args": ["runserver", "0.0.0.0:8001"],
      "django": true,
      "env": {
        "DEBUG": "True"
      }
    }
  ]
}
```

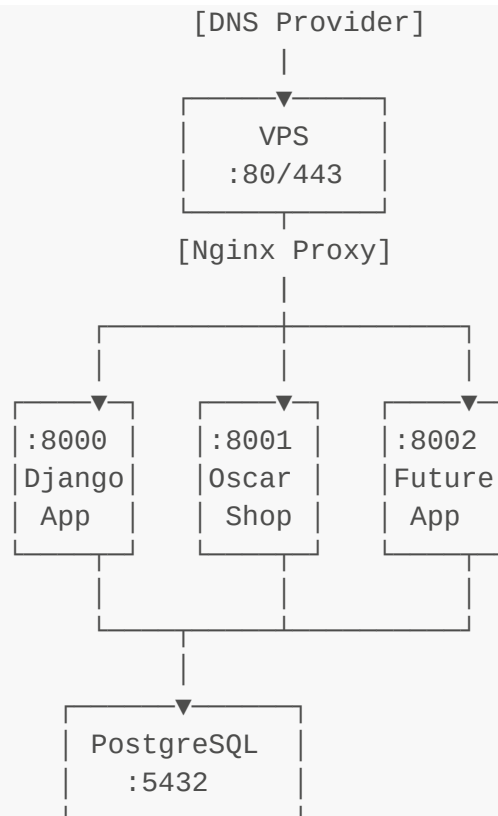
Port Forwarding

```
# En VSCode > Terminal > PORTS:
# Forward port 8001 para testing local
# Acceder desde navegador: localhost:8001
```

FASE 7: MULTI-APLICACIONES

7.1 Arquitectura Multi-App

```
 Arquitectura VPS:
                         Internet
                        |
```



7.2 Configuración Django Oscar

```
# Crear estructura para Oscar Shop
mkdir -p /home/deploy/oscar_shop
cd /home/deploy/oscar_shop

# Crear proyecto Oscar (futuro)
python3 -m venv venv
source venv/bin/activate
pip install django-oscar

# Configurar Nginx para Oscar (puerto 8001)
sudo tee /etc/nginx/sites-available/tienda.arasmu.net << 'EOF'
server {
    listen 80;
    server_name tienda.arasmu.net;
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl http2;
    server_name tienda.arasmu.net;

    # SSL certificates
    ssl_certificate /etc/letsencrypt/live/tienda.arasmu.net/fullchain.pem;
    ssl_certificate_key
/etc/letsencrypt/live/tienda.arasmu.net/privkey.pem;
```

```

location / {
    proxy_pass http://127.0.0.1:8001;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
}
EOF

```

7.3 Gestión de Puertos

🔊 Mapa de Puertos:

🌐 Público:

- └─ Puerto 80 (HTTP) → Nginx → Redirect a 443
- └─ Puerto 443 (HTTPS) → Nginx → Proxy reverso
- └─ Puerto 22 (SSH) → Acceso administrativo

🏠 Interno:

- └─ Puerto 8000 → Django Ecodisseny
- └─ Puerto 8001 → Django Oscar Shop
- └─ Puerto 8002 → Future App
- └─ Puerto 5432 → PostgreSQL

📊 FASE 8: MONITOREO Y MANTENIMIENTO

8.1 Scripts de Monitoreo

```

# Script de monitoreo general
cat > /home/deploy/scripts/monitor.sh << 'EOF'
#!/bin/bash
echo "=== 📊 ESTADO DEL SERVIDOR $(date) ==="
echo "📊 CPU y Memoria:"
echo "CPU: $(top -bn1 | grep "Cpu(s)" | awk '{print $2}' | cut -d'%' -f1)%
usado"
echo "RAM: $(free -h | awk 'NR==2{printf "%.1f/%.1fGB (%.1f%%)\n",
$3/1024/1024,$2/1024/1024,$3*100/$2}')"
echo "💾 Espacio: $(df -h / | tail -1 | awk '{print $3 "/" $2 " (" $5
")"}')'"

echo "🔧 Servicios:"
for service in nginx ecodisseny postgresql; do
    if systemctl is-active $service >/dev/null; then
        echo "✅ $service: ACTIVO"
    else
        echo "❌ $service: INACTIVO"
    fi
done

```

```
echo "🌐 Web: $(curl -s -o /dev/null -w "%{http_code}"  
https://app.arasmu.net)"  
EOF  
  
chmod +x /home/deploy/scripts/monitor.sh
```

8.2 Backups Automatizados

```
# Script de backup completo  
cat > /home/deploy/scripts/backup_completo.sh << 'EOF'  
#!/bin/bash  
DATE=$(date +%Y%m%d_%H%M%S)  
BACKUP_DIR="/home/deploy/backups"  
  
# Backup BD  
pg_dump ecodisseny_prod > $BACKUP_DIR/ecodisseny_db_$DATE.sql  
gzip $BACKUP_DIR/ecodisseny_db_$DATE.sql  
  
# Backup media  
tar -czf $BACKUP_DIR/ecodisseny_media_$DATE.tar.gz -C  
/home/deploy/ecodisseny media/  
  
# Limpiar antiguos (7 días)  
find $BACKUP_DIR -name "ecodisseny_*" -mtime +7 -delete  
  
echo "✅ Backup completado: $DATE"  
EOF  
  
chmod +x /home/deploy/scripts/backup_completo.sh  
  
# Cron para backup diario  
(crontab -l 2>/dev/null; echo "0 2 * * *  
/home/deploy/scripts/backup_completo.sh") | crontab -
```

8.3 Scripts de Actualización

```
# Script de actualización  
cat > /home/deploy/scripts/actualizar_app.sh << 'EOF'  
#!/bin/bash  
cd /home/deploy/ecodisseny  
  
# Backup de seguridad  
/home/deploy/scripts/backup_completo.sh  
  
# Actualizar código  
git pull origin docker  
source venv/bin/activate  
pip install -r requirements.txt
```

```
# Aplicar cambios
python manage.py migrate --settings=ecodisseny.settings_production
python manage.py cargar_documentacion --update --
settings=ecodisseny.settings_production
python manage.py collectstatic --noinput --
settings=ecodisseny.settings_production

# Reiniciar servicios
sudo systemctl restart ecodisseny
sudo systemctl reload nginx

echo "✓ Actualización completada"
EOF

chmod +x /home/deploy/scripts/actualizar_app.sh
```

🔍 FASE 9: TROUBLESHOOTING

9.1 Problemas Comunes

Error 502 Bad Gateway

```
# Verificar Gunicorn
sudo systemctl status ecodisseny
sudo journalctl -u ecodisseny -f

# Verificar Nginx
sudo nginx -t
sudo systemctl restart ecodisseny nginx
```

Error de Base de Datos

```
# Verificar PostgreSQL
sudo systemctl status postgresql

# Test de conexión
python -c "
import psycopg2
conn = psycopg2.connect(
    host='localhost',
    database='ecodisseny_prod',
    user='deploy',
    password='tu_password'
)
print('✓ Conexión exitosa')
"
```


Usar Scripts de Configuración

```
# Si hay problemas, usar script de reinicio completo
./setup_complete.sh

# O recargar fixtures y usuarios
./load_fixtures.sh
```

9.2 Debugging Avanzado

```
# Ver logs en tiempo real
sudo tail -f /var/log/nginx/app.arasmu.net.error.log \
            /var/log/django/ecodisseny.log

# Verificar procesos
ps aux | grep gunicorn
sudo netstat -tlnp | grep :8000

# Verificar espacio
df -h
free -h
```

9.3 Checklist de Verificación

- ✓ VPS Contabo contratado y configurado
- ✓ Usuario deploy con permisos sudo
- ✓ Firewall configurado (22, 80, 443)
- ✓ PostgreSQL funcionando
- ✓ Repositorio clonado
- ✓ Scripts setup_complete.sh ejecutado
- ✓ Usuarios y perfiles creados
- ✓ Nginx configurado
- ✓ DNS Cloudflare configurado
- ✓ SSL Let's Encrypt funcionando
- ✓ Aplicación accesible vía HTTPS
- ✓ VSCode SSH configurado
- ✓ Backups automatizados
- ✓ Scripts de monitoreo funcionando

 ¡DESPLIEGUE COMPLETADO!

✓ Resultado Final:

-  Aplicación Django en <https://app.arasmu.net>

-  **HTTPS seguro** con certificados automáticos
-  **Backups diarios** automatizados
-  **Monitoreo** del sistema
-  **VSCode remoto** para desarrollo
-  **Base preparada** para Oscar Shop

Usuarios Listos:

 **ADMIN:**

- mulastone / ecodisseny2024
- gonzalo / ecodisseny2024

 **USUARIOS:**

- sarah / ecodisseny2024
- pilar / ecodisseny2024
- santiago / ecodisseny2024
- roger / ecodisseny2024

Próximos Pasos:

1.  Instalar Django Oscar Shop (puerto 8001)
2.  Configurar monitoreo avanzado
3.  CI/CD para despliegues automáticos
4.  Notificaciones por email
5.  Personalización y branding

¡Tu aplicación está lista para producción! 